

3. Optimiseurs : SGD, MomentumSGD et AdamW

Les trois variantes modifient la même information de base : le gradient courant.

$$\text{SGD: } \theta_{t+1} = \theta_t - \eta g_t$$

$$\text{Momentum: } v_{t+1} = \mu v_t - \eta g_t, \quad \theta_{t+1} = \theta_t + v_{t+1}$$

$$\text{AdamW: } \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \lambda \theta_t$$

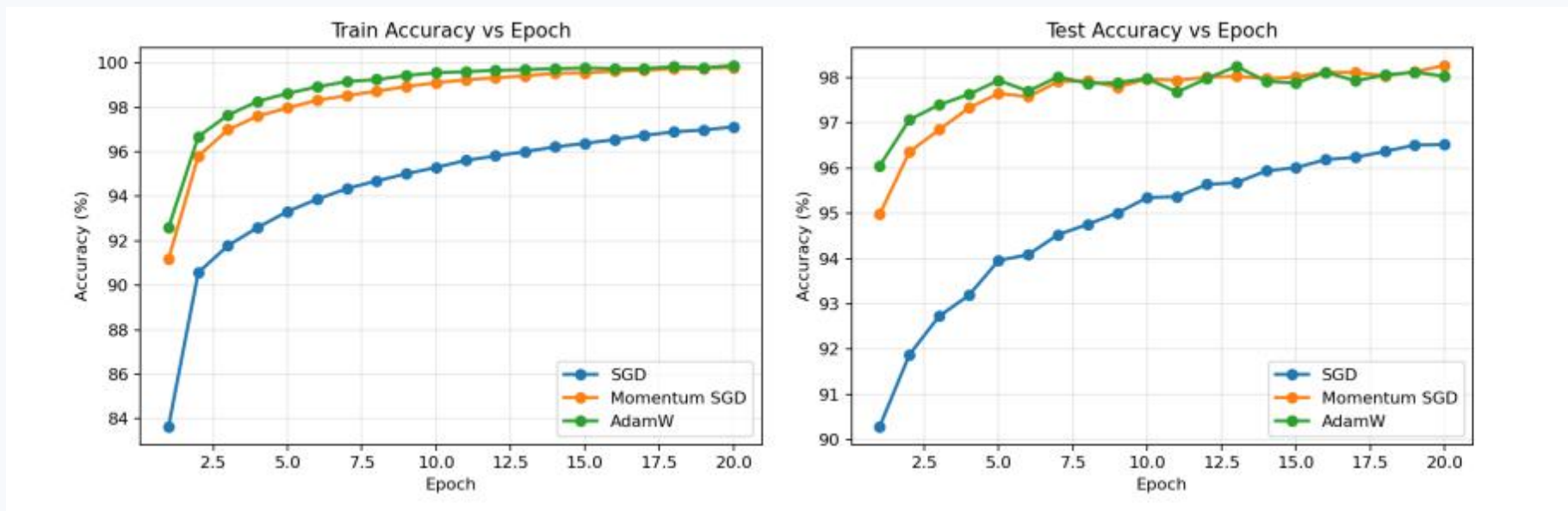
SGD converge plus lentement et reste en retrait.

MomentumSGD atteint la meilleure précision test : 98,27%.

AdamW est très proche : 98,03%, avec un démarrage rapide.

3. Boucle d'entraînement commune

Trainer::runEpoch(...) regroupe apprentissage et évaluation pour MLP, CNN, ResNet et modes distribués.



Comparaison des précisions d'entraînement/test pour trois optimiseurs sur le MLP

AdamW converge rapidement grâce à son learning rate adaptatif et reste très utilisé dans les réseaux modernes.

5. MaxPooling : réduction spatiale et extraction des caractéristiques

Exemple

Entrée :

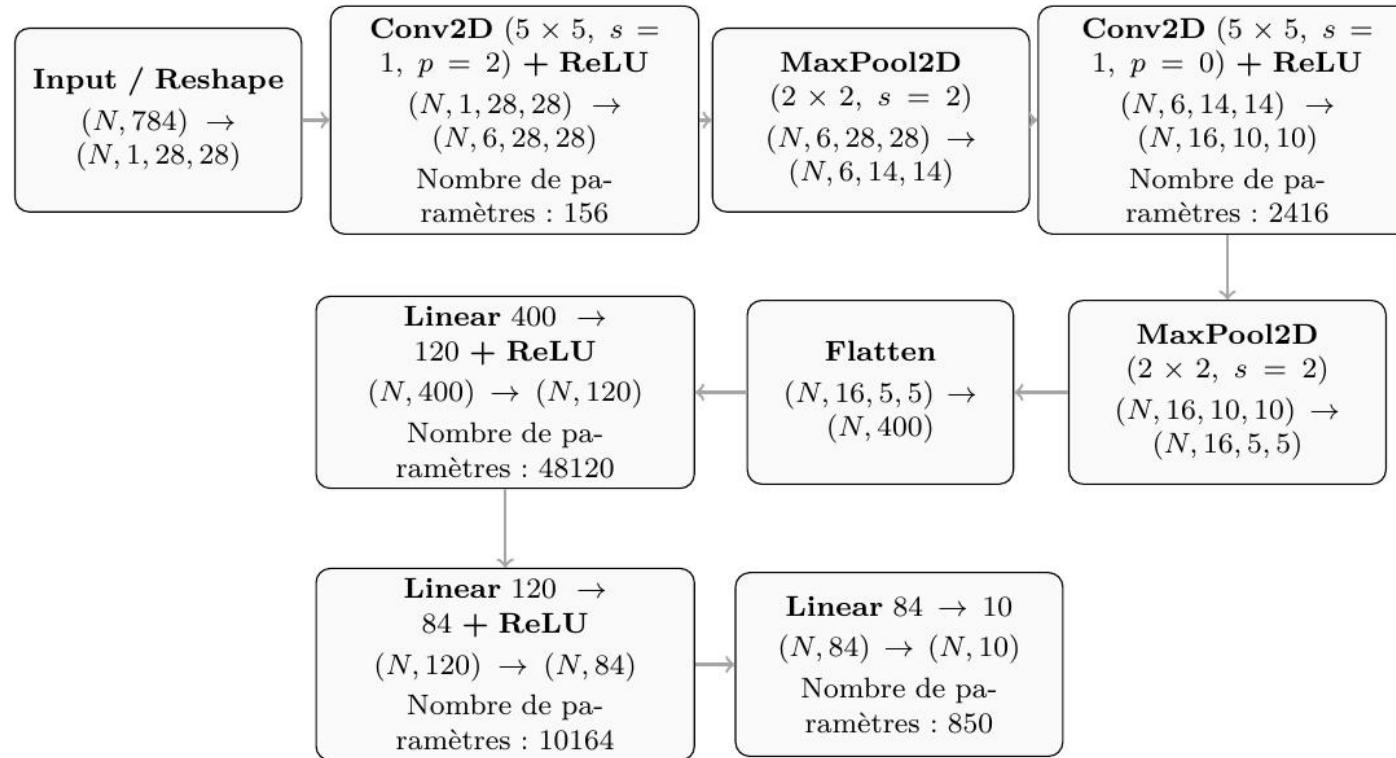
$$\begin{bmatrix} 1 & 2 \\ 5 & 3 \end{bmatrix} \rightarrow 5$$

Réduit la taille des cartes de caractéristiques

Diminue le coût de calcul des couches suivantes

5. Structure et flux de données du CNN actuel

Chemin inspiré de LeNet : deux convolutions, deux pooling, puis une tête entièrement connectée.

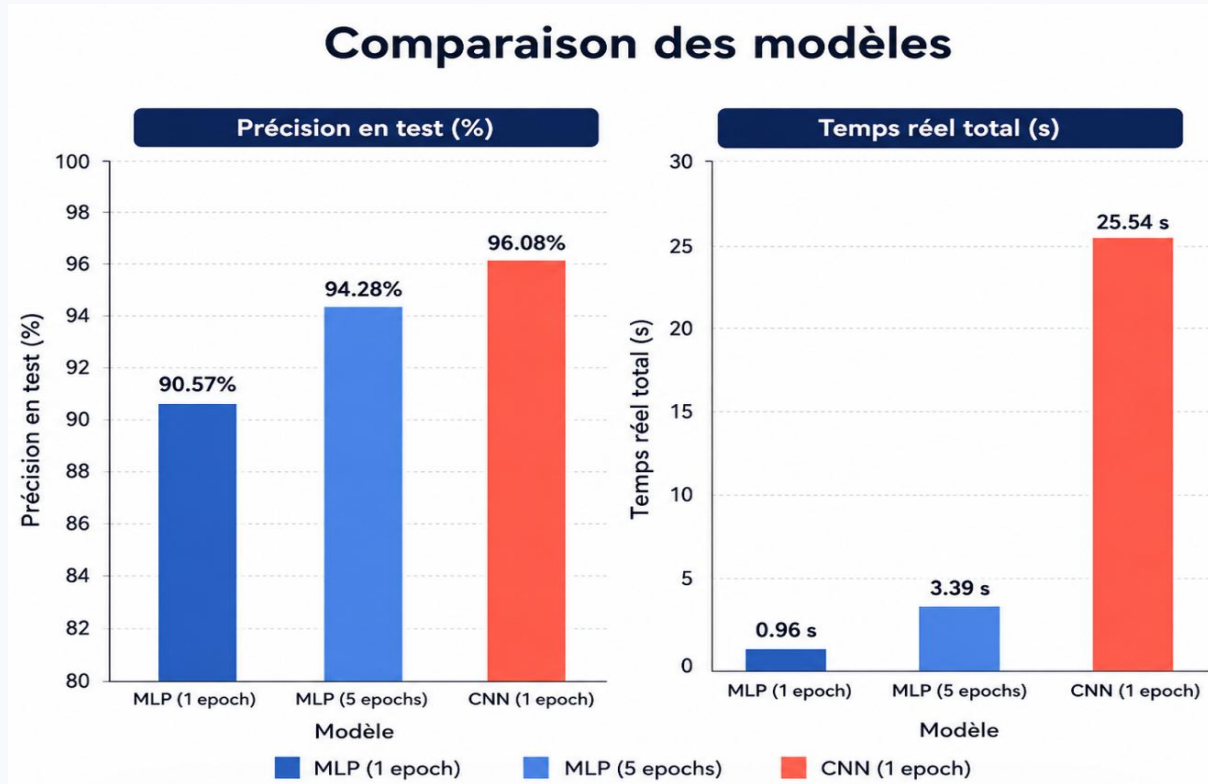


Nombre total de paramètres entraînaibles : 61706

évolution des shapes et nombre de paramètres : 61 706 paramètres entraînaibles.

5. Vérification à l'exécution : précision vs coût

Le CNN valide le chemin convolutif, mais son coût est beaucoup plus élevé que celui du MLP.



$$\text{accuracy} = \frac{\# \text{ prédictions correctes}}{\# \text{ exemples}}$$

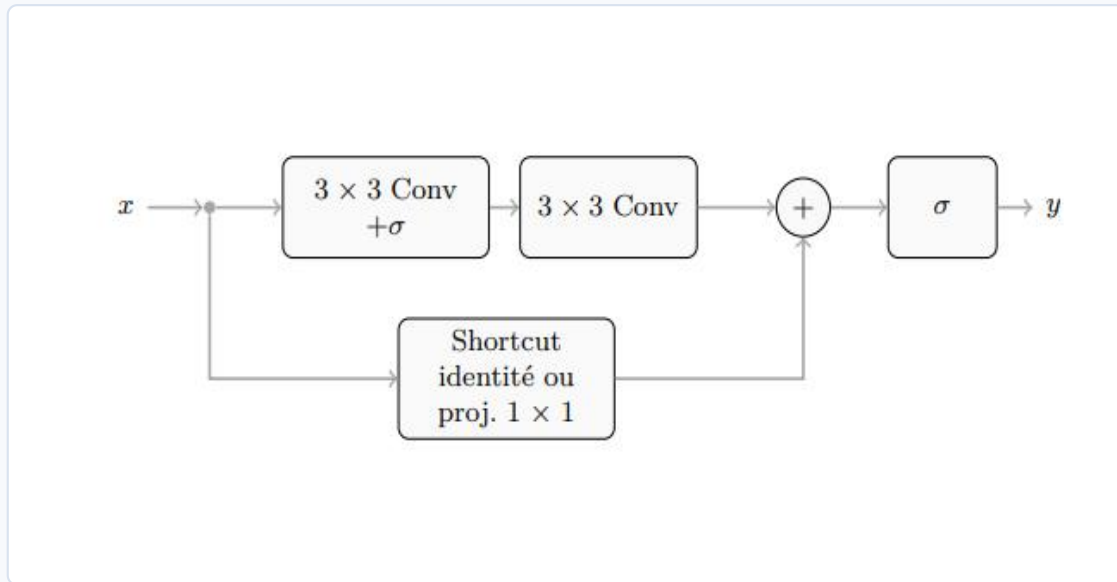
$$\text{ratio de coût} = \frac{T_{CNN,1e}}{T_{MLP,1e}} = \frac{25.54}{0.96} \approx 26.6 \times$$

$$\frac{T_{CNN,1e}}{T_{MLP,5e}} = \frac{25.54}{3.39} \approx 7.5 \times$$

Cette vérification sert de base avant ResNet et Tiny-ImageNet.

5. Extension vers une structure ResNet

La connexion résiduelle apprend une correction plutôt qu'un mapping complet.



$$H(x) = x + F(x), \quad y = F(x) + P(x)$$

Stages : 16 -> 32 -> 64 canaux ;

résolution 28×28 -> 14×14 -> 7×7 .

Intérêt : réseau plus profond, optimisation plus stable.

6. Optuna : recherche automatique d'hyperparamètres

Rôle : remplacer une exploration manuelle coûteuse par des trials pilotés et comparables.

$$\text{score} = \begin{cases} \max_e \text{test_acc}_e, & \text{metric} = \text{best} \\ \text{test_acc}_E, & \text{metric} = \text{last} \end{cases} \quad \text{max score}$$

Espace exploré : learning rate, batch size, activation, initialisation et optimiseur.

40 trials × 3 epochs

best test_acc = 0,9891

Gain observé

+0,46 point vs défaut manuel